

Écris ton process. L'agent l'exécute.

À la place d'un canevas de boîtes et de flèches : une note qui décrit le travail en français, un agent qui la lit et l'exécute avec de vrais outils, un script pour les chiffres. Tout est ici, à copier.

- Ce qu'on a mis en place au lieu d'un outil d'automatisation.
- L'arborescence complète, et le fichier d'instructions de l'agent.
- Une note de process, puis la fiche de méthode qui en découle.
- Le garde-fou, et les trois cas où le canevas reste le bon choix.
- Le texte de première exécution, à lancer sous tes yeux.

Aucune ligne de code à écrire : des fichiers texte, et deux fichiers à copier tels quels.

Ce qu'on a fait à la place d'un canevas.

Un canevas d'automatisation tient tant que les données sont propres. Le jour où l'entrée devient un courriel mal écrit, un site d'entreprise ou une note de réunion, les boîtes se multiplient et plus personne n'ose y toucher.

Le canevas dessiné	Le process écrit
Une boîte par cas particulier	Une phrase par cas, dans une note
Se modifie en cliquant, sans trace	Se modifie en corrigeant une phrase datée
Personne ne sait plus ce qu'il fait	Se lit en une minute, par un humain
Le jugement se code en conditions	Le jugement revient à l'agent, cadré par la note

Ce que ça ne remplace pas

Un flux régulier sur des données propres, entre deux outils du marché, reste plus simple en canevas. La page 10 donne les trois cas où on ne discute pas.

Chez nous, chaque process est une note : préparation d'appel, rappel des demandes, publication. Quand il change, on corrige une phrase.

Trois pièces, à la place du canevas.

Ce que fait le canevas

Applique des règles fixes

Échoue sur une entrée imparfaite

Se modifie en cliquant

Ce que fait l'agent

Lit, compare, choisit selon la note

Demande ou signale ce qui manque

Se modifie en corrigeant une phrase

La note

Le process écrit en français : quoi, pour qui, dans quel ordre, ce qui est interdit, ce qu'on rend à la fin.

L'agent

Il lit la note et fait le travail avec de vrais outils : fichiers, terminal, pages web. Il juge ce qui doit être jugé.

Le script

Les chiffres sortent d'un programme, jamais du modèle. Un total faux est pire qu'un total absent.

Le script reste indispensable : un modèle ne compte pas, il estime. Tout total passe par un programme.

Où vivent les process.

Un dossier, quelques fichiers texte. Les process dans leur propre sous-dossier : c'est ce qui permet d'en ajouter sans rien casser.

Deux règles de rangement

Un process par fichier, jamais deux. Et rien dans les notes de base qui ne vaille pour tous les process : les détails descendent dans le process concerné.

dossier/

```
dossier/
├── CLAUDE.md          ← instructions de l'agent
├── 00-qui-on-est.md
├── 01-objectifs.md
├── 02-environnement.md
├── 03-regles.md
├── Process/
│   ├── <process-1>.md  ← un process = une note
│   └── <process-2>.md
├── Projets/<nom>.md
├── .claude/
│   ├── settings.json
│   ├── hooks/garde-fou.sh
│   └── skills/<nom>/SKILL.md
└── scripts/<mesure>.py
```

Les clés restent hors du dossier, dans un fichier protégé. Rien de secret ne descend dans une note.

Tu peux commencer avec un seul process et deux notes de base. Le reste s'ajoute quand le besoin arrive.

Le fichier que l'agent lit en premier.

Il dit dans quel ordre lire, comment travailler, et ce qui est interdit. C'est le seul fichier que l'agent ouvre sans qu'on le lui demande.

CLAUDE.md

Instructions de l'agent

Lis d'abord, dans cet ordre :
00-qui-on-est.md, 01-objectifs.md,
02-environnement.md, 03-regles.md.

Avant tout travail : reformule la demande en une phrase, liste les étapes, puis exécute.

Un process se trouve dans Process/<nom>.md.
Suis-le ; s'il manque, dis-le, ne l'invente pas.

Interdit sans un humain : dépenser, envoyer à un tiers, publier, supprimer, créer un compte.

Les chiffres sortent d'un script, jamais de toi.

Fin de tâche : note l'état, les décisions et les problèmes ouverts dans Projets/<nom>.md.

« S'il manque, dis-le, ne l'invente pas » est la ligne qui évite les process imaginaires, rédigés avec assurance.

« Reformule la demande en une phrase » révèle les malentendus avant qu'ils coûtent une heure de travail.

À quoi ressemble une note de process.

Prends la tâche que tu refais le plus souvent. Écris-la comme tu l'expliquerais à quelqu'un qui commence demain.

La bonne longueur

Une page. Un process de trois pages cache en réalité deux process qu'il faut séparer.

Ce qui rend une note utile

« Interdit » et « fini quand ». Sans eux, l'agent improvise la fin et tu relis tout.

Process/<nom>.md

<nom du process>

Quand : <la situation qui le déclenche>

Pour qui : <le destinataire du résultat>

Étapes :

1. <ce qu'on lit, et où>
2. <ce qu'on écrit, et où>
3. <ce qu'on vérifie avant de rendre>

Interdit : <ce qui ne se fait jamais ici>

Rendu attendu : <le fichier ou le message, avec son format>

Fini quand : <le contrôle qui le prouve>

Quand le process revient, on en fait une fiche.

Une fiche de méthode, c'est un process rangé de façon que l'agent l'ouvre tout seul quand la situation correspond. Sa description dit quand l'utiliser, pas seulement ce qu'elle fait.

Où mettre la fiche

Un dossier par fiche, avec son fichier de méthode. L'agent les liste au démarrage et n'ouvre que celle qui correspond.

Quand la créer

Après deux exécutions propres.
Avant, tu figes des erreurs que tu n'as pas encore vues.

```
.claude/skills/<nom>/SKILL.md
```

```
---
```

```
name: fiche-appel
```

```
description: Préparer la fiche d'une entreprise  
avant un appel. Utiliser quand on parle de  
préparer un appel ou une fiche prospect.
```

```
---
```

```
# Fiche d'appel
```

1. Lis le site et la fiche publique de l'entreprise.
2. Remplis Process/fiche-appel.md : activité, taille, ce que dit son site, une question à poser.
3. Une ligne par champ, en français.
4. Introuvable : écris « inconnu », n'invente pas.
5. N'appelle jamais, n'envoie rien.
6. Termine par : trouvé, pas trouvé, temps passé.

La description est ce qui déclenche l'ouverture. « Utiliser quand... » vaut mieux que « ce skill sert à... ».

Deux exécutions à la main avant d'automatiser.

On n'automatise pas un process qu'on n'a jamais fait tourner. Ce texte sert à la première exécution, sous tes yeux.

1

Première exécution

Tu regardes, tu corriges la note là où l'agent a hésité. La plupart des corrections portent sur ce qui était implicite pour toi.

2

Deuxième exécution

Sur un autre cas. Si elle passe sans correction, le process est stable : tu peux en faire une fiche de méthode.

`prompts/premiere-execution.md`

`Lis Process/<nom>.md.`

`Exécute-le pour : <le cas concret d'aujourd'hui>.`

`Sources autorisées : <les pages, les fichiers>.`

`Ne modifie aucun fichier avant de m'avoir
montré le résultat complet.`

`Termine par trois lignes :`

- `- ce que tu as trouvé,`
- `- ce que tu n'as pas trouvé,`
- `- le temps passé.`

Ce que l'agent ne fera jamais.

La note dit ce qui est interdit ; ce fichier le fait respecter. Il lit chaque commande avant qu'elle parte et refuse celles que tu as listées.

Ce qu'il protège

Ce qui ne se rattrape pas :
l'historique réécrit, les fichiers
supprimés, les secrets lus par
mégarde.

Ce qu'il ne fait pas

Juger la qualité du travail. Un
garde-fou empêche les
catastrophes, il ne relit pas à ta
place.

`.claude/hooks/garde-fou.sh`

```
#!/bin/bash
cmd=$(python3 -c 'import json, sys
d = json.load(sys.stdin).get("tool_input", {})
print(d.get("command", ""))')
case "$cmd" in
  *"push --force"*|"reset --hard"*|\
  *"rm -rf"*|*".env"*)
    echo "Refusé par le garde-fou : $cmd" >&2
    exit 2 ;;
esac
exit 0
```

Ajoute tes propres interdits entre les barres verticales : ce sont tes gestes irréversibles à toi.

Un garde-fou se teste : demande une commande interdite et vérifie le refus. Tant qu'il n'a rien refusé, il n'existe pas.

Trois cas où le canevas reste le bon choix.

Le test rapide

Si tu peux écrire toutes les conditions à l'avance, prends le canevas. Sinon, écris la note.

Le cas mixte

Rien n'interdit les deux : un canevas pour la collecte régulière, une note et un agent pour ce qui demande de lire et décider.

La situation	Pourquoi le canevas gagne
Un flux répétitif sur des champs propres	aucun jugement : des règles fixes suffisent
Deux outils du marché déjà connectés	l'intégration existe, la brancher prend dix minutes
Un coût d'exécution à rendre prévisible	chaque exécution coûte le même prix, au centime

La frontière, en une phrase

Dès qu'une étape demande de lire quelque chose d'imparfait et de décider, la note et l'agent passent devant. Tant que tout est propre et régulier, le canevas fait très bien l'affaire.

La question n'est jamais « quel outil est meilleur », mais « cette étape demande-t-elle du jugement ». Réponds étape par étape.

Trois phrases à garder.

- Le process s'écrit en français, il ne se dessine plus en boîtes.
- L'agent fait le jugement, le script fait les chiffres, tu décides.
- Deux exécutions à la main avant toute automatisisation.
- Un process de trois pages en cache deux : sépare-les.

Par où commencer

La tâche que tu refais chaque semaine. Écris-la ce soir, exécute-la demain avec l'agent.

Le piège

Écrire dix process d'un coup. Un seul, exécuté deux fois, vaut mieux que dix jamais lancés.

Quand tu veux aller plus loin

Ce que tu ajoutes

Le même process revient chaque semaine

une fiche de méthode, qui s'ouvre toute seule

Les chiffres deviennent importants

un script dédié, appelé par la note

Plusieurs personnes exécutent

un rendu attendu identique, écrit dans la note

Quel process vas-tu écrire en premier ? Réponds à mon message, je te dis par quelle étape le commencer.